```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no, viewport-fit=cover">
<title>GEOMETRY MINI</title>
<style>
  *{margin:0;padding:0;box-sizing:border-box}

body{background:#0a0a14;color:white;font-family:monospace;overflow:hidden;touch-action:none;position:fixed;width:100%;height:100%}

canvas{display:block;width:100vw;height:100vh;background:linear-gradient(#111a2a,#1a102a)}

.screen{position:absolute;inset:0;display:none;flex-direction:column;align-items:center;justify-content:center;padding:20px;z-index:20}
  .screen.show{display:flex}
  #menu{background:linear-gradient(180deg,#0a0a14,#1a102a)}
  .logo{font-size:52px;font-weight:900;text-align:center;line-height:.85}
  .logo span{color:#00ff88;text-shadow:0 0 20px #00ff88}
  .subtitle{opacity:.6;margin:12px 0 28px;font-size:12px;letter-spacing:3px}
  .big-btn{width:100%;max-width:320px;padding:18px;border-radius:16px;border:2px solid #00ff88;background:#00ff88;color:#000;font-size:20px;font-weight:900;margin-bottom:12px;cursor:pointer}
  .big-btn.secondary{background:transparent;color:white;border-color:#444}
  .player-card{background:#1a1a2a;border:1px solid #333;border-radius:14px;padding:12px 16px;display:flex;gap:12px;align-items:center;margin-top:16px}
  .cube{width:32px;height:32px;background:#00ff88;border-radius:6px}
  .listScreen{background:#0a0a14;justify-content:flex-start;overflow-y:auto}

.topbar{width:100%;max-width:420px;display:flex;justify-content:space-between;align-items:center;margin-bottom:12px;gap:8px}
  .topbar button{background:#222;color:#fff;border:1px solid #555;padding:10px 14px;border-radius:10px;white-space:nowrap}
  .tabs{width:100%;max-width:420px;display:flex;gap:8px;margin-bottom:12px}
  .tabs button{flex:1;padding:12px;border-radius:12px;border:1px solid #333;background:#1a1a2a;color:#fff;font-weight:bold}
  .tabs button.on{background:#00ff88;color:#000;border-color:#00ff88}

.slots{width:100%;max-width:420px;display:flex;flex-direction:column;gap:10px;padding-bottom:20px}
  .slot{background:#1a1a2a;border:2px solid #333;border-radius:14px;padding:16px 14px;display:flex;justify-content:space-between;align-items:center;cursor:pointer;min-height:58px}
  .slot.filled{border-color:#00ff88}
  .slot.community{border-color:#ff8800}
```

```
  .slot.cloud{border-color:#44aaff}
  .slot
b{font-size:17px;white-space:nowrap;overflow:hidden;text-overflow:ellipsis;max-width:260px}
  .slot .arrow{font-size:20px;flex-shrink:0}
  .tag{background:#222;border:1px solid #444;padding:4px
8px;border-radius:20px;font-size:10px}
  .tag.green{border-color:#00ff88;color:#00ff88}
  .tag.blue{border-color:#44aaff;color:#44aaff}
  .sticky-actions{position:sticky;top:0;z-index:30;background:#0a0a14;padding:10px 0
12px;display:flex;gap:8px;width:100%;max-width:420px}
  .sticky-actions
button{flex:1;padding:16px;border-radius:14px;font-weight:900;font-family:monospace;cursor
:pointer;font-size:16px}
  .btn-play{background:#00ff88;color:#000;border:2px solid #00ff88}
  .btn-like{background:#1a1a2a;color:#fff;border:2px solid #ff66ff}
  .btn-like.liked{background:#ff66ff;color:#000}
  .detail-card{width:100%;max-width:420px;background:#1a1a2a;border:2px solid
#333;border-radius:20px;padding:18px}
  .detail-card.community{border-color:#ff8800}
  .detail-card.mine{border-color:#00ff88}
  .detail-card.cloud{border-color:#44aaff}
  .detail-row{display:flex;justify-content:space-between;padding:8px 0;border-bottom:1px
solid #222;font-size:13px}
  .detail-row b{color:#00ff88}

.completer{display:flex;align-items:center;gap:10px;background:#111;padding:10px;border-ra
dius:12px;margin-bottom:8px}
  .completer .mini{width:28px;height:28px;border-radius:6px}
  .vote-bar{display:flex;align-items:center;gap:8px;margin-bottom:6px;font-size:12px}
  .vote-bar .label{width:80px}
  .vote-bar .track{flex:1;height:10px;background:#222;border-radius:10px;overflow:hidden}
  .vote-bar .fill{height:100%}
  .vote-btns{display:flex;gap:6px;flex-wrap:wrap;margin:10px 0}
  .vote-btns button{flex:1;min-width:70px;padding:10px;border-radius:10px;border:1px solid
#444;background:#222;color:#fff;cursor:pointer;font-size:12px}
  .vote-btns
button.voted{border-color:#00ff88;background:#00ff88;color:#000;font-weight:900}
  .vote-btns button.locked{opacity:.35;border-style:dashed;cursor:not-allowed}

#topInfo{position:absolute;top:0;left:0;right:0;height:36px;background:rgba(0,0,0,0.85);displa
y:flex;align-items:center;justify-content:center;gap:8px;font-size:12px;z-index:20;white-space
:nowrap;overflow:hidden;padding:0 8px}

#floatActions{position:absolute;top:44px;right:8px;display:none;flex-direction:column;gap:8px
;z-index:40}
  #gameScreen.editor-mode #floatActions{display:flex}
```

```
  .float-btn{width:62px;height:62px;border-radius:50%;border:3px solid
#fff;font-size:24px;cursor:pointer;display:flex;align-items:center;justify-content:center;box-sha
dow:0 4px 12px rgba(0,0,0,0.6);font-family:monospace}
  .float-settings{background:#00ff88}
  .float-publish{background:#44aaff}

.float-label{position:absolute;top:44px;right:78px;display:none;flex-direction:column;gap:14px
;z-index:40;align-items:flex-end;font-size:10px;font-weight:900;text-align:right}
  #gameScreen.editor-mode .float-label{display:flex}
  .float-label span{background:rgba(0,0,0,0.85);padding:4px
8px;border-radius:8px;height:20px;display:flex;align-items:center;margin-top:22px}
  .float-label span:first-child{margin-top:0}

#editorToolbar{position:absolute;left:0;right:0;bottom:132px;height:52px;background:rgba(0,0
,0,0.92);display:flex;gap:6px;align-items:center;padding:6px
8px;overflow-x:auto;overflow-y:hidden;z-index:20;scrollbar-width:none}
  #editorToolbar::-webkit-scrollbar{display:none}
  #editorToolbar button{flex:0 0
auto;min-width:68px;height:40px;background:#222;color:#fff;border:1px solid
#555;border-radius:10px;font-family:monospace;font-size:11px;cursor:pointer;padding:4px
8px}
  #editorToolbar button.active{background:#ff4444}
  #editorToolbar button.edit-active{background:#00ff88;color:#000}
  #editorToolbar button.move-active{background:#44aaff;color:#000;font-weight:900}
  #editorToolbar
button.publish-btn{background:#00ff88;color:#000;font-weight:900;border-color:#00ff88;min-
width:110px}
  #editorToolbar button.publish-btn.locked{background:#333;color:#999;border-color:#555}

#playToolbar{position:absolute;left:0;right:0;bottom:0;height:52px;background:rgba(0,0,0,0.9
2);display:none;gap:8px;align-items:center;justify-content:center;padding:6px;z-index:20}
  #playToolbar
button{flex:1;max-width:200px;height:40px;background:#222;color:#fff;border:1px solid
#555;border-radius:10px;font-family:monospace;cursor:pointer}

#paleta{position:absolute;bottom:0;left:0;width:100%;background:rgba(0,0,0,0.95);padding:8
px;display:grid;grid-template-columns:repeat(6,1fr);gap:6px;z-index:10}
  #gameScreen.editor-mode #paleta{bottom:184px}
  #gameScreen.editor-mode #editorToolbar{display:flex}
  #gameScreen.play-mode #playToolbar{display:flex}

.b{height:52px;border-radius:8px;display:flex;flex-direction:column;align-items:center;justify-c
ontent:center;font-size:9px;font-weight:bold;border:3px solid transparent}
  .b.sel{border-color:#00ff88;transform:scale(1.05)}

#hint{position:absolute;bottom:190px;left:50%;transform:translateX(-50%);background:#000
a;padding:6px 12px;border-radius:20px;font-size:11px;white-space:nowrap;z-index:10}
```

```
  #ghost{position:absolute;width:40px;height:40px;border:2px dashed
#00ff88;border-radius:6px;pointer-events:none;display:none;z-index:15}

#settingsModal{position:absolute;inset:0;background:rgba(0,0,0,0.85);display:none;align-ite
ms:center;justify-content:center;z-index:60;padding:20px}
  #settingsModal.show{display:flex}
  .modalBox{background:#1a1a2a;border:2px solid
#00ff88;border-radius:20px;padding:20px;width:100%;max-width:360px;max-height:85vh;ov
erflow-y:auto}
  .modalBox
button{width:100%;padding:14px;margin-bottom:8px;border-radius:12px;border:1px solid
#444;background:#222;color:#fff;cursor:pointer;font-family:monospace}
  .modalBox button.publish{background:#00ff88;color:#000;font-weight:900}
  .modalBox button.danger{background:#ff444422;border-color:#ff4444;color:#ff8888}

.cloud-status{width:100%;max-width:420px;padding:10px;border-radius:12px;font-size:12px;t
ext-align:center;margin-bottom:10px}
  .cloud-status.on{background:#00ff8822;border:1px solid #00ff88}
  .cloud-status.off{background:#ff444422;border:1px solid #ff4444}
  .cloud-status.loading{background:#ffcc0022;border:1px solid #ffcc00}
  .empty-state{text-align:center;opacity:.6;padding:30px 10px;line-height:1.8;font-size:13px}
  .empty-state b{color:#00ff88;font-size:16px}
</style>
</head>
<body>

<div id="menu" class="screen show">
  <div class="logo">GEOMETRY<br><span>MINI</span></div>
  <div class="subtitle">CREA • JUEGA • COMPARTE</div>
  <button class="big-btn" onclick="UI.show('createScreen')">✏️ CREAR</button>
  <button class="big-btn secondary" onclick="UI.show('playScreen')">▶️ JUGAR</button>
  <div class="player-card"><div class="cube" id="playerCube"></div><div><b
id="playerNameTxt">Jugador</b><br><small id="playerStatsTxt" style="opacity:.6">0
niveles • 0 muertes</small></div><button style="background:#222;color:#fff;border:1px solid
#555;padding:8px;border-radius:8px" onclick="PLAYER.rename()">✏️</button></div>
  <div style="margin-top:14px;font-size:10px;opacity:.4">GEOMETRY MINI v3.9 • Versión
Limpia para Compartir</div>
  <div id="menuCloudStatus" style="margin-top:10px;font-size:11px;opacity:.7">☁️
Conectando a la nube...</div>
</div>

<div id="createScreen" class="screen listScreen">
  <div class="topbar"><button onclick="UI.show('menu')">⬅️ VOLVER</button><b>MIS
NIVELES</b><span style="font-size:11px;opacity:.5">5 slots</span></div>
  <div
style="width:100%;max-width:420px;font-size:11px;opacity:.6;margin-bottom:10px">Tocá un
nivel para entrar. Para publicarlo, entrá y usá el botón azul 🚀 o el ⚙️ de arriba a la
derecha.</div>
```

```
  <div class="slots" id="slotsList"></div>
</div>

<div id="playScreen" class="screen listScreen">
  <div class="topbar"><button onclick="UI.show('menu')">⬅️ VOLVER</button><b>🎮
JUGAR</b><span style="font-size:11px;opacity:.6">v3.9 ☁️</span></div>
  <div id="cloudStatus" class="cloud-status loading">☁️ Conectando a la nube...</div>
  <button onclick="CLOUD.fetchAll()"
style="width:100%;max-width:420px;padding:12px;border-radius:12px;background:#44aaff;c
olor:#000;border:none;font-weight:900;font-family:monospace;margin-bottom:10px;cursor:po
inter">🔄 ACTUALIZAR DESDE LA NUBE</button>
  <div class="tabs"><button id="tabAll" class="on" onclick="PLAY.setTab('all')">🌍
TODOS</button><button id="tabMine" onclick="PLAY.setTab('mine')">👤 MIS
NIVELES</button></div>
  <div style="width:100%;max-width:420px;display:flex;gap:8px;margin-bottom:12px"><input
id="searchInput" placeholder="Buscar por nombre..." maxlength="15"
oninput="PLAY.search(this.value)" style="flex:1;background:#1a1a2a;border:1px solid
#333;color:#fff;padding:12px;border-radius:12px;font-family:monospace"><span
id="searchCount" style="font-size:11px;opacity:.5;align-self:center">0/15</span></div>
  <div style="width:100%;max-width:420px;font-size:11px;opacity:.6;margin-bottom:10px"
id="playInfo"></div>
  <div class="slots" id="playList"></div>
</div>

<div id="detailScreen" class="screen"
style="justify-content:flex-start;overflow-y:auto;padding-top:10px">
  <div class="topbar"><button onclick="UI.show('playScreen')">⬅️
SALIR</button><b>DETALLES</b><span></span></div>
  <div class="sticky-actions">
    <button class="btn-play" onclick="DETAIL.play()">▶️ PLAY</button>
    <button class="btn-like" id="likeBtn" onclick="DETAIL.toggleLike()">🤍 LIKE</button>
  </div>
  <div class="detail-card" id="detailCard">
    <div style="text-align:center;margin-bottom:8px"><div id="detailIcon"
style="font-size:42px">🟩</div><h2 id="detailName" style="font-size:24px;margin:6px
0">Nivel</h2><div id="detailAuthor" style="opacity:.6;font-size:13px">Por Autor</div><div
id="detailPublished" style="margin-top:6px"></div></div>
    <div class="detail-row"><span>🎯 Dificultad votada</span><b
id="detailDiff">Fácil</b></div>
    <div class="detail-row"><span>🧱 Bloques</span><b id="detailBlocks">0</b></div>
    <div class="detail-row"><span>❤️ Likes</span><b id="detailLikes">0</b></div>
    <div class="detail-row"><span>💀 Intentos</span><b id="detailAttempts">0</b></div>
    <div class="detail-row"><span>✅ Completado por</span><b
id="detailCompletersCount">0</b></div>
    <div class="detail-row"><span>📅 Creado</span><b id="detailDate">Hoy</b></div>
    <div style="margin:16px 0 8px"><b>📊 ¿QUÉ DIFICULTAD LE PONES?</b><br><small
style="opacity:.6">Votá una vez. Inhumano reservado.</small></div>
    <div id="voteBars"></div>
```

```
    <div class="vote-btns" id="voteBtns">
      <button data-vote="Fácil" onclick="DETAIL.vote('Fácil')">😊 Fácil</button>
      <button data-vote="Medio" onclick="DETAIL.vote('Medio')">😐 Medio</button>
      <button data-vote="Difícil" onclick="DETAIL.vote('Difícil')">😤 Difícil</button>
      <button data-vote="Imposible" onclick="DETAIL.vote('Imposible')">💀
Imposible</button>
      <button class="locked">🔒 INHUMANO</button>
    </div>
    <div style="margin:12px 0 8px"><b>👤 CREADOR</b></div>
    <div class="completer"><div class="mini" id="creatorCube"
style="background:#00ff88"></div><div><b id="creatorName">Autor</b><br><small
id="creatorInfo" style="opacity:.6">Creador</small></div></div>
    <div style="margin:12px 0 8px"><b>🏆 LO COMPLETARON</b></div>
    <div id="completersList"></div>
    <button class="big-btn secondary" onclick="UI.show('playScreen')">⬅️ SALIR</button>
  </div>
</div>

<div id="gameScreen" class="screen" style="padding:0">
  <div id="topInfo"><span>💀 <b id="deaths">0</b></span><span>|</span><span
id="modeTxt">CUBO</span><span>|</span><span
id="editTxt">JUGANDO</span><span>|</span><span id="slotName">NIVEL</span></div>
  <div class="float-label"><span>⚙️ AJUSTES</span><span>🚀 PUBLICAR</span></div>
  <div id="floatActions">
    <button class="float-btn float-settings" onclick="GAME.openSettings()">⚙️</button>
    <button class="float-btn float-publish" id="floatPublishBtn"
onclick="GAME.publishFromToolbar()">🚀</button>
  </div>
  <div id="ghost"></div>
  <canvas id="game"></canvas>
  <div id="hint">Toca para saltar</div>
  <div id="editorToolbar">
    <button onclick="GAME.exit()">⬅️ SALIR</button>
    <button id="btnEdit">✏️ EDITOR</button>
    <button id="btnMove">✋ MOVER</button>
    <button id="btnSave">💾 GUARDAR</button>
    <button id="btnErase">🧹 BORRAR</button>
    <button onclick="GAME.openSettings()">⚙️ AJUSTES</button>
    <button id="toolbarPublishBtn" class="publish-btn"
onclick="GAME.publishFromToolbar()">🚀 PUBLICAR ☁️</button>
  </div>
  <div id="playToolbar"><button onclick="GAME.exitToPlay()">⬅️ SALIR</button><button
onclick="GAME.restart()">🔄 REINTENTAR</button></div>
  <div id="paleta"></div>
  <div id="settingsModal">
    <div class="modalBox">
      <h3 style="text-align:center;margin-bottom:12px">⚙️ AJUSTES - <span
id="settingsName">Nivel</span></h3>
```

```
      <div id="settingsInfo"
style="font-size:12px;opacity:.7;line-height:1.6;margin-bottom:12px"></div>
      <button onclick="GAME.renameFromSettings()">✏️ RENOMBRAR NIVEL</button>
      <button id="settingsPublishBtn" class="publish"
onclick="GAME.publishFromSettings()">🚀 PUBLICAR EN LA NUBE ☁️</button>
      <button onclick="GAME.shareCode()">📤 COMPARTIR POR WHATSAPP</button>
      <button onclick="GAME.duplicateFromSettings()">📋 DUPLICAR NIVEL</button>
      <button class="danger" onclick="GAME.deleteFromSettings()">🗑️ BORRAR
NIVEL</button>
      <button onclick="GAME.closeSettings()" style="background:#333">❌
CERRAR</button>
    </div>
  </div>
</div>

<script>
const CLOUD = {
  BASE: 'https://geometry-mini-default-rtdb.firebaseio.com',
  cache: [], connected: false,
  async fetchAll(){
    let status=document.getElementById('cloudStatus');
    if(status) { status.className='cloud-status loading'; status.innerText='☁️ Bajando niveles
de la nube...'; }
    try{
      let res=await fetch(this.BASE+'/levels.json');
      if(!res.ok) throw new Error('no ok');
      let data=await res.json();
      this.cache=[];
      if(data){
        Object.keys(data).forEach(k=>{
          let l=data[k];
          if(l && l.level && Array.isArray(l.level)){ l.id='cloud-'+k; l.isCloud=true; l.isMine=false;
this.cache.push(l); }
        });
      }
      this.connected=true;
      if(status){ status.className='cloud-status on'; status.innerText='☁️ Nube conectada •
'+this.cache.length+' niveles de la comunidad'; }
      let menuStatus=document.getElementById('menuCloudStatus'); if(menuStatus)
menuStatus.innerText='☁️ Nube conectada • '+this.cache.length+' niveles en línea';
      PLAY.render();
    }catch(e){
      this.connected=false;
      if(status){ status.className='cloud-status off'; status.innerText='⚠️ Sin internet / sin
nube. Mostrando solo niveles locales. Tocá de nuevo para reintentar.'; }
    }
  },
  async publish(slot){
```

```
    let cleanName = slot.name.toLowerCase().replace(/[^a-z0-9]/g,'_').slice(0,15) || 'nivel';
    let cloudId = cleanName + '_' + Date.now();
    let payload = { name: slot.name, author: PLAYER.get().name, authorColor:
PLAYER.get().color, blocks: slot.level.length, difficulty: DIFFICULTY.calc(slot.votes), likes:
slot.likes||0, attempts: slot.attempts||0, completers: slot.completers||[], date: new
Date().toLocaleDateString(), published:true, verified:true, votes: slot.votes, level: slot.level,
cloudId: cloudId, owner: PLAYER.get().name };
    let res=await fetch(this.BASE+'/levels/'+cloudId+'.json', { method:'PUT', body:
JSON.stringify(payload), headers:{'Content-Type':'application/json'} });
    if(!res.ok) throw new Error('No se pudo subir');
    slot.cloudId=cloudId; slot.published=true; DB.save();
    await this.fetchAll();
    return cloudId;
  },
  async unpublish(slot){
    if(!slot.cloudId) { slot.published=false; DB.save(); return; }
    await fetch(this.BASE+'/levels/'+slot.cloudId+'.json', { method:'DELETE' });
    slot.published=false; slot.cloudId=null; DB.save();
    await this.fetchAll();
  }
};
const DB = {
  KEY:'geometry_mini_v39', BACKUP_KEY:'geometry_mini_v39_backup', data:null,
  load(){
    try{ let raw=localStorage.getItem(this.KEY); if(!raw) throw 0; let d=JSON.parse(raw);
if(!d||!Array.isArray(d.slots)||d.slots.length!==5) throw 0;
      d.slots.forEach(s=>{ if(!Array.isArray(s.level)) s.level=[]; s.likes=s.likes||0;
s.liked=s.liked||false; s.attempts=s.attempts||0; s.completers=s.completers||[];
s.published=s.published||false; s.verified=s.verified||false;
s.votes=s.votes||{'Fácil':0,'Medio':0,'Difícil':0,'Imposible':0}; s.myVote=s.myVote||null;
s.cloudId=s.cloudId||null; });
      this.data=d;
    }catch(e){ try{ let b=JSON.parse(localStorage.getItem(this.BACKUP_KEY));
if(b&&b.slots){ this.data=b; this.save(); return this.data; } }catch(_){}
      this.data={ version:'3.9', player:{ name:'Jugador', color:'#00ff88', totalDeaths:0 },
slots:Array.from({length:5},(_,i)=>({ id:i, name:'Nivel '+(i+1), level:[], deaths:0, updated:0,
empty:true, likes:0, liked:false, attempts:0, completers:[], published:false, verified:false,
votes:{'Fácil':0,'Medio':0,'Difícil':0,'Imposible':0}, myVote:null, cloudId:null })) };
      this.save();
    } return this.data;
  },
  save(){ try{ localStorage.setItem(this.BACKUP_KEY, localStorage.getItem(this.KEY)||'');
localStorage.setItem(this.KEY, JSON.stringify(this.data)); }catch(e){} },
  resetAll(){ if(confirm('¿Borrar los 5 niveles?')){ localStorage.removeItem(this.KEY);
this.load(); UI.renderSlots(); UI.renderMenu(); } }
};
const PLAYER = {
  get(){ return DB.data.player; },
```

```
  rename(){ let n=prompt('Tu nombre de jugador (máx 15)', this.get().name); if(n){
n=n.slice(0,15); this.get().name=n; DB.save(); UI.renderMenu(); UI.renderSlots();
PLAY.render(); } },
  addDeath(){ this.get().totalDeaths++; }
};
// v3.9 - SIN NIVELES DE PRUEBA. La comunidad se construye con los niveles reales que
publiquen tus amigos.
const COMMUNITY = { levels: [] };
const DIFFICULTY = {
  order:['Fácil','Medio','Difícil','Imposible','Inhumano'],

colors:{'Fácil':'#00ff88','Medio':'#ffcc00','Difícil':'#ff8800','Imposible':'#ff4444','Inhumano':'#aa0
0ff'},
  votable:['Fácil','Medio','Difícil','Imposible'],
  calc(votes){ let max=0, win='Fácil'; for(let k of this.votable){ if((votes[k]||0)>max){
max=votes[k]; win=k; } } return win; },
  total(votes){ return this.votable.reduce((a,k)=>a+(votes[k]||0),0); }
};
const SEARCH = {
  normalize(s){ return (s||'').toLowerCase().trim().slice(0,15); },
  filter(list,q){ let qq=this.normalize(q); if(!qq) return list; return list.filter(l=>
this.normalize(l.name).includes(qq) || this.normalize(l.author).includes(qq)); }
};
const UI = {
  show(id){
    document.querySelectorAll('.screen').forEach(s=>s.classList.remove('show'));
document.getElementById(id).classList.add('show');
    if(id==='createScreen') this.renderSlots(); if(id==='menu') this.renderMenu();
if(id==='playScreen') PLAY.render();
  },
  renderMenu(){ let p=PLAYER.get();
document.getElementById('playerNameTxt').innerText=p.name;
document.getElementById('playerCube').style.background=p.color; let
f=DB.data.slots.filter(s=>!s.empty).length;
document.getElementById('playerStatsTxt').innerText=f+'/5 niveles • '+p.totalDeaths+'
muertes'; },
  renderSlots(){
    let list=document.getElementById('slotsList'); list.innerHTML='';
    DB.data.slots.forEach((s,i)=>{
      let div=document.createElement('div'); div.className='slot'+(s.empty?'':'
filled')+(s.published?' published':'');
      div.innerHTML=`<b>${s.name}</b><span class="arrow">▶️</span>`;
      div.onclick=()=>GAME.openEditor(i); list.appendChild(div);
    });
  }
};
const PLAY = {
  tab:'all', query:'',
```

```
  setTab(t){ this.tab=t; document.getElementById('tabAll').classList.toggle('on',t==='all');
document.getElementById('tabMine').classList.toggle('on',t==='mine'); this.render(); },
  search(v){ this.query=v; document.getElementById('searchCount').innerText=v.length+'/15';
this.render(); },
  getAll(){
    let mine = DB.data.slots.filter(s=>!s.empty).map(s=>({ id:'mine-'+s.id, name:s.name,
author:PLAYER.get().name, authorColor:PLAYER.get().color, blocks:s.level.length,
difficulty:DIFFICULTY.calc(s.votes), likes:s.likes||0, liked:s.liked||false, attempts:s.attempts||0,
completers:s.completers||[], date:s.updated?new
Date(s.updated).toLocaleDateString():'Hoy', level:s.level, empty:s.empty, isMine:true,
isCloud:false, slotId:s.id, published:s.published, verified:s.verified, votes:s.votes,
myVote:s.myVote }));
    let comm = COMMUNITY.levels.map(c=>({ ...c, isMine:false, isCloud:false }));
    let cloud = CLOUD.cache.map(c=>({ ...c, isMine:false, isCloud:true, liked:false,
myVote:null }));
    let cloudFiltered = cloud.filter(c=> !(c.owner===PLAYER.get().name &&
mine.some(m=>m.name===c.name)));
    if(this.tab==='mine') return mine;
    return [...comm, ...cloudFiltered, ...mine.filter(m=>m.published)];
  },
  render(){
    let list=document.getElementById('playList'); list.innerHTML='';
    let all=this.getAll(); let filtered=SEARCH.filter(all, this.query);
    if(this.tab==='mine'){
      document.getElementById('playInfo').innerText = `Tus niveles: ${all.length}/5`;
    } else {
      document.getElementById('playInfo').innerText = all.length===0 ? 'Aún no hay niveles
publicados. ¡Sé el primero!' : `${all.length} niveles en la comunidad ☁️`;
    }
    if(filtered.length===0){
      if(all.length===0 && !this.query){
        list.innerHTML='<div class="empty-state"><b>🌍 La nube está vacía</b><br>¡Sé el
primero en publicar!<br><br>1. Andá a CREAR<br>2. Hacé tu nivel<br>3. Pasátelo para
verificarlo<br>4. Tocá 🚀 PUBLICAR ☁️<br><br>Tus amigos lo van a ver acá.</div>';
      } else {
        list.innerHTML='<div style="text-align:center;opacity:.5;padding:30px">No hay niveles
con ese nombre.<br>Probá ACTUALIZAR DESDE LA NUBE</div>';
      }
      return;
    }
    filtered.forEach(l=>{
      let div=document.createElement('div'); div.className='slot'+(l.isCloud?' cloud':l.isMine?'
filled':' community');
      div.innerHTML=`<b>${l.isCloud?'☁️ ':l.isMine?'🟩 ':'🟧 '}${l.name}</b><span
class="arrow">▶️</span>`;
      div.onclick=()=>DETAIL.open(l); list.appendChild(div);
    });
  }
```

```
};
const DETAIL = {
  current:null,
  open(levelObj){
    this.current=levelObj;
    document.getElementById('detailName').innerText=levelObj.name;
    document.getElementById('detailAuthor').innerText='Por '+levelObj.author;

document.getElementById('detailIcon').innerText=levelObj.isCloud?'☁️':levelObj.isMine?'🟩':
'🟧';
    document.getElementById('detailDiff').innerText=levelObj.difficulty;
    document.getElementById('detailBlocks').innerText=levelObj.blocks;
    document.getElementById('detailLikes').innerText=levelObj.likes;
    document.getElementById('detailAttempts').innerText=levelObj.attempts||0;

document.getElementById('detailCompletersCount').innerText=(levelObj.completers||[]).lengt
h+' personas';
    document.getElementById('detailDate').innerText=levelObj.date||'Hoy';
    document.getElementById('detailPublished').innerHTML= levelObj.published ? '<span
class="tag green">🌍 PUBLICADO</span> <span class="tag green">✅
VERIFICADO</span>' : (levelObj.isCloud?'<span class="tag blue">☁️ DE LA
NUBE</span>':'<span class="tag">🔒 NO PUBLICADO</span>');
    document.getElementById('creatorName').innerText=levelObj.author;

document.getElementById('creatorCube').style.background=levelObj.authorColor||'#00ff88';
    document.getElementById('creatorInfo').innerText=levelObj.isCloud?'Nivel de la nube de
GEOMETRY MINI':levelObj.isMine?'Tu nivel':'Creador de la comunidad';
    let card=document.getElementById('detailCard'); card.className='detail-card
'+(levelObj.isCloud?'cloud':levelObj.isMine?'mine':'community');
    let list=document.getElementById('completersList'); list.innerHTML='';
    if(!levelObj.completers || levelObj.completers.length===0) list.innerHTML='<div
style="opacity:.5;font-size:12px;padding:8px">Nadie lo completó todavía. ¡Sé el
primero!</div>';
    else levelObj.completers.forEach(c=>{ let d=document.createElement('div');
d.className='completer'; d.innerHTML=`<div class="mini"
style="background:${c.color}"></div><div><b>${c.name}</b><br><small
style="opacity:.6">✅ Completó</small></div><span style="margin-left:auto">🏆</span>`;
list.appendChild(d); });
    this.renderVotes(); this.updateLikeBtn(); UI.show('detailScreen');
    document.getElementById('detailScreen').scrollTop=0;
  },
  renderVotes(){
    let votes=this.current.votes||{'Fácil':0,'Medio':0,'Difícil':0,'Imposible':0};
    let total=DIFFICULTY.total(votes); let bars=document.getElementById('voteBars');
bars.innerHTML='';
    DIFFICULTY.order.forEach(k=>{
      let count = k==='Inhumano' ? 0 : (votes[k]||0);
      let pct = total>0 ? Math.round(count/total*100) : 0;
```

```
      let div=document.createElement('div'); div.className='vote-bar';
      div.innerHTML=`<span class="label">${k==='Inhumano'?'🔒 Inhumano':k}</span><div
class="track"><div class="fill"
style="width:${pct}%;background:${DIFFICULTY.colors[k]}"></div></div><span
style="width:50px;text-align:right">${count} (${pct}%)</span>`;
      bars.appendChild(div);
    });
    document.querySelectorAll('#voteBtns button[data-vote]').forEach(b=>{
b.classList.toggle('voted', b.dataset.vote===this.current.myVote); });
  },
  vote(choice){
    if(!DIFFICULTY.votable.includes(choice)) return;
    let votes=this.current.votes;
    if(this.current.myVote){
votes[this.current.myVote]=Math.max(0,(votes[this.current.myVote]||0)-1); }
    votes[choice]=(votes[choice]||0)+1; this.current.myVote=choice;
this.current.difficulty=DIFFICULTY.calc(votes);
    document.getElementById('detailDiff').innerText=this.current.difficulty;
    if(this.current.isMine){ let s=DB.data.slots[this.current.slotId]; s.votes=votes;
s.myVote=choice; DB.save(); }
    this.renderVotes();
  },
  updateLikeBtn(){ let b=document.getElementById('likeBtn'); let liked=this.current.liked;
b.innerText=liked?'❤️ LIKED • '+this.current.likes:'🤍 LIKE • '+this.current.likes;
b.classList.toggle('liked', !!liked); },
  toggleLike(){
    this.current.liked=!this.current.liked; this.current.likes+= this.current.liked?1:-1;
    if(this.current.isMine){ let s=DB.data.slots[this.current.slotId]; s.likes=this.current.likes;
s.liked=this.current.liked; DB.save(); }
    this.updateLikeBtn(); document.getElementById('detailLikes').innerText=this.current.likes;
PLAY.render();
  },
  play(){ GAME.openPlay(this.current); },
  onComplete(levelObj){
    if(!levelObj.completers) levelObj.completers=[];
    let me=PLAYER.get().name;
    if(!levelObj.completers.find(c=>c.name===me)) levelObj.completers.push({name:me,
color:PLAYER.get().color});
    if(levelObj.isMine){ let s=DB.data.slots[levelObj.slotId]; s.completers=levelObj.completers;
s.verified=true; DB.save(); UI.renderSlots(); }
  }
};
const TYPES = { 1:{n:'BLOQUE',c:'#fff',txt:'#000'}, 2:{n:'PINCHO',c:'#ff4444',txt:'#fff'},
3:{n:'ORBE ROSA',c:'#ff66ff',txt:'#fff'}, 4:{n:'ORBE AMAR',c:'#ffff00',txt:'#000'}, 5:{n:'ORBE
AZUL',c:'#44aaff',txt:'#fff'}, 6:{n:'PORTAL CUBO',c:'#00ff88',txt:'#000'}, 7:{n:'PORTAL
NAVE',c:'#ff8800',txt:'#000'}, 8:{n:'GRAV ABAJO',c:'#8888ff',txt:'#fff'}, 9:{n:'GRAV
ARRIBA',c:'#ff88ff',txt:'#000'}, 10:{n:'PAD ROSA',c:'#ff66ff',txt:'#fff'}, 11:{n:'PAD
AMAR',c:'#ffff00',txt:'#000'}, 12:{n:'PAD AZUL',c:'#44aaff',txt:'#fff'} };
```

```
const GAME = {
  canvas:null, ctx:null, deaths:0, camX:0, camY:0, isEditor:false, selectedType:1,
eraseMode:false, moveMode:false, currentSlot:0, mode:'editor', currentPlayLevel:null,
hasCompleted:false,
  level:[],
player:{x:100,y:350,w:30,h:30,vy:0,mode:'CUBE',grav:1,prevY:350,prevX:100,onGround:fals
e,coyoteTimer:0,bufferTimer:0}, input:{press:false,pressConsumed:false}, lastTime:0, acc:0,
triggers:new Set(), portalCooldown:0,
  gesture:{ state:'IDLE', startX:0, startY:0, startTime:0, moved:false, lastPlace:0 },
  init(){
    this.canvas=document.getElementById('game'); this.ctx=this.canvas.getContext('2d');
    window.addEventListener('resize',()=>this.resize()); this.resize();
    let pal=document.getElementById('paleta');
    Object.keys(TYPES).forEach(k=>{ let d=document.createElement('div');
d.className='b'+(k==1?' sel':''); d.style.background=TYPES[k].c; d.style.color=TYPES[k].txt;
d.innerHTML='<span
style="font-size:18px">'+(k==2?'▲':k<=5?'●':k<=9?'◉':'▬')+'</span>'+TYPES[k].n;
d.onclick=()=>{this.selectedType=parseInt(k); this.setErase(false); this.setMove(false);
document.querySelectorAll('.b').forEach(b=>b.classList.remove('sel')); d.classList.add('sel');};
pal.appendChild(d); });
    this.bindInput(); requestAnimationFrame(t=>this.loop(t));
  },
  resize(){ this.canvas.width=window.innerWidth; this.canvas.height=window.innerHeight; },
  updatePublishButtons(){
    let s=DB.data.slots[this.currentSlot]; if(!s) return;
    let toolbarBtn=document.getElementById('toolbarPublishBtn');
    let floatBtn=document.getElementById('floatPublishBtn');
    if(s.published){ if(toolbarBtn) toolbarBtn.innerText='🌍 DESPUBLICAR'; if(floatBtn)
floatBtn.innerText='🌍'; }
    else if(s.verified){ if(toolbarBtn){ toolbarBtn.innerText='🚀 PUBLICAR ☁️';
toolbarBtn.classList.remove('locked'); } if(floatBtn) floatBtn.innerText='🚀'; }
    else { if(toolbarBtn){ toolbarBtn.innerText='🔒 PUBLICAR';
toolbarBtn.classList.add('locked'); } if(floatBtn) floatBtn.innerText='🔒'; }
    let settingsBtn=document.getElementById('settingsPublishBtn');
    if(settingsBtn) settingsBtn.innerText = s.published ? '🌍 DESPUBLICAR DE LA NUBE' :
(s.verified ? '🚀 PUBLICAR EN LA NUBE ☁️' : '🔒 PUBLICAR (pásatelo primero)');
  },
  openEditor(i){
    this.mode='editor'; this.currentSlot=i;
this.level=JSON.parse(JSON.stringify(DB.data.slots[i].level||[]));
    if(this.level.length===0)
this.level=[{t:1,x:500,y:350},{t:1,x:540,y:350},{t:2,x:620,y:370},{t:11,x:720,y:370}];
    this.deaths=DB.data.slots[i].deaths||0;
document.getElementById('deaths').innerText=this.deaths;
    document.getElementById('slotName').innerText=DB.data.slots[i].name.toUpperCase();
    document.getElementById('gameScreen').className='screen show editor-mode';
```

```
    document.getElementById('editorToolbar').style.display='flex';
document.getElementById('playToolbar').style.display='none';
document.getElementById('paleta').style.display='grid';
    this.camX=0; this.camY=0; this.moveMode=false; this.setMove(false);
    UI.show('gameScreen'); document.getElementById('gameScreen').className='screen
show editor-mode';
    this.resetPlayer(); this.isEditor=false; this.gesture.state='IDLE';
    document.getElementById('hint').innerText='✏️ EDITOR para construir | 🚀 azul arriba =
PUBLICAR';
    this.updatePublishButtons();
  },
  openPlay(levelObj){
    this.mode='play'; this.currentPlayLevel=levelObj;
this.level=JSON.parse(JSON.stringify(levelObj.level||[])); this.hasCompleted=false;
    this.deaths=0; document.getElementById('deaths').innerText='0';
    document.getElementById('slotName').innerText=(levelObj.name+' •
'+levelObj.author).toUpperCase();
    document.getElementById('gameScreen').className='screen show play-mode';
    document.getElementById('editorToolbar').style.display='none';
document.getElementById('playToolbar').style.display='flex';
document.getElementById('paleta').style.display='none';
    if(levelObj.isMine){ let s=DB.data.slots[levelObj.slotId]; s.attempts=(s.attempts||0)+1;
DB.save(); }
    UI.show('gameScreen'); document.getElementById('gameScreen').className='screen
show play-mode';
    this.resetPlayer(); this.isEditor=false;
  },
  exit(){ if(this.mode==='play'){ this.exitToPlay(); return; } if(confirm('¿Salir? Si no guardaste,
se pierden los cambios.')) UI.show('createScreen'); },
  exitToPlay(){ UI.show('playScreen'); },
  restart(){ this.hasCompleted=false; this.resetPlayer(); },
  save(){ let s=DB.data.slots[this.currentSlot]; s.level=this.level; s.deaths=this.deaths;
s.empty=false; s.updated=Date.now(); DB.save(); UI.renderMenu(); UI.renderSlots();
this.updatePublishButtons(); alert('✅ Guardado en '+s.name); },
  setErase(v){ this.eraseMode=v; if(v) this.setMove(false); let
b=document.getElementById('btnErase'); b.innerText=v?'🧹 ON':'🧹 BORRAR';
b.classList.toggle('active',v); },
  setMove(v){ this.moveMode=v; if(v) this.setErase(false); let
b=document.getElementById('btnMove'); b.innerText=v?'✋ MOVER ON':'✋ MOVER';
b.classList.toggle('move-active',v); document.getElementById('hint').innerText=v?'✋
MOVER: arrastrá con 1 dedo':'✏️ 1 toque = 1 bloque'; },
  openSettings(){
    let s=DB.data.slots[this.currentSlot];
    document.getElementById('settingsName').innerText=s.name;
    document.getElementById('settingsInfo').innerHTML=`<b>Creador:</b>
${PLAYER.get().name}<br><b>Fecha:</b> ${s.updated?new
Date(s.updated).toLocaleDateString():'Sin guardar'}<br><b>Bloques:</b>
${this.level.length}<br><b>Estado:</b> ${s.verified?'✅ Verificado':'⚠️ Sin verificar - tenés
```

```
que pasarte el nivel'} • ${s.published?'🌍 Publicado en la nube ☁️':'🔒 No
publicado'}<br><b>Likes:</b> ${s.likes} • <b>Completaron:</b>
${s.completers.length}<br><b>Dificultad:</b> ${DIFFICULTY.calc(s.votes)}`;
    this.updatePublishButtons();
    document.getElementById('settingsModal').classList.add('show');
  },
  closeSettings(){ document.getElementById('settingsModal').classList.remove('show'); },
  renameFromSettings(){ let s=DB.data.slots[this.currentSlot]; let n=prompt('Nuevo nombre
(máx 15)', s.name); if(n){ n=n.slice(0,15); s.name=n; DB.save();
document.getElementById('slotName').innerText=n.toUpperCase();
document.getElementById('settingsName').innerText=n; this.openSettings(); } },
  async publishFromToolbar(){ await this.doPublish(); },
  async publishFromSettings(){ await this.doPublish(); this.openSettings(); },
  async doPublish(){
    let s=DB.data.slots[this.currentSlot];
    s.level=this.level; s.empty=false; s.updated=Date.now(); DB.save();
    if(s.published){
      if(confirm('¿Despublicar '+s.name+' de la nube?')){
        try{ await CLOUD.unpublish(s); alert('🔒 Despublicado'); this.updatePublishButtons();
}catch(e){ alert('Error al despublicar: '+e.message); }
      }
      return;
    }
    if(!s.verified){
      if(confirm('🔒 Todavía no está verificado.\n\nPara publicar tenés que pasarte el nivel
primero.\n\n¿Querés jugarlo ahora para verificarlo?')){
        this.isEditor=false; document.getElementById('editTxt').innerText='JUGANDO';
this.resetPlayer(); this.camX=0;
      }
      return;
    }
    try{
      let btn=document.getElementById('toolbarPublishBtn'); if(btn) btn.innerText='☁️
SUBIENDO...';
      await CLOUD.publish(s);
      alert('✅ ¡PUBLICADO EN LA NUBE!\n\nAhora en el otro celu: JUGAR > ACTUALIZAR
DESDE LA NUBE y aparece "'+s.name+'" con el ☁️');
      this.updatePublishButtons();
    }catch(e){ alert('❌ Error al subir.\n'+e.message); this.updatePublishButtons(); }
  },
  shareCode(){
    let s=DB.data.slots[this.currentSlot];
    let
code='GM-'+btoa(unescape(encodeURIComponent(JSON.stringify({n:s.name,a:PLAYER.get
().name,l:s.level})))).slice(0,200);
    let text='Mirá mi nivel de GEOMETRY MINI: '+s.name+' por
'+PLAYER.get().name+'\nCódigo: '+code;
```

```
    if(navigator.share){ navigator.share({text:text}); } else { prompt('Copiá este código y
mandalo por WhatsApp:', text); }
  },
  duplicateFromSettings(){
    let emptyIdx=DB.data.slots.findIndex(x=>x.empty);
    if(emptyIdx===-1){ alert('No hay slot vacío'); return; }
    let s=DB.data.slots[this.currentSlot];
    DB.data.slots[emptyIdx]={...JSON.parse(JSON.stringify(s)), id:emptyIdx, name:s.name+'
copia', published:false, cloudId:null, updated:Date.now()};
    DB.save(); alert('✅ Duplicado en slot '+(emptyIdx+1)); this.closeSettings();
  },
  deleteFromSettings(){ if(confirm('¿Borrar '+DB.data.slots[this.currentSlot].name+'?')){ let
s=DB.data.slots[this.currentSlot]; if(s.published && s.cloudId){ CLOUD.unpublish(s); }
DB.data.slots[this.currentSlot]={id:this.currentSlot,name:'Nivel
'+(this.currentSlot+1),level:[],deaths:0,updated:0,empty:true,likes:0,liked:false,attempts:0,co
mpleters:[],published:false,verified:false,votes:{'Fácil':0,'Medio':0,'Difícil':0,'Imposible':0},myV
ote:null,cloudId:null}; DB.save(); this.closeSettings(); UI.show('createScreen'); } },
  resetPlayer(){ let p=this.player; p.x=100; p.y=350; p.vy=0; p.mode='CUBE'; p.grav=1;
this.triggers.clear(); p.bufferTimer=0; p.coyoteTimer=0;
document.getElementById('modeTxt').innerText='CUBO'; if(this.mode==='play') this.camX=0;
},
  die(){ this.deaths++; PLAYER.addDeath(); this.resetPlayer(); },
  tryPlace(wx,wy){
    if(this.mode!=='editor' || !this.isEditor) return;
    if(this.moveMode) return;
    let now=Date.now(); if(now - this.gesture.lastPlace < 120) return;
    this.gesture.lastPlace=now;
    let gx=Math.floor(wx/40)*40, gy=Math.floor(wy/40)*40;
    if(gy<0 || gy>400) return;
    if(this.eraseMode){ this.level=this.level.filter(o=>{ let dx=(o.x+20)-wx, dy=(o.y+20)-wy;
return Math.sqrt(dx*dx+dy*dy)>45; }); return; }
    if(this.level.some(o=>o.x===gx && o.y===gy && o.t===this.selectedType)) return;
    this.level=this.level.filter(o=>!(o.x===gx&&o.y===gy));
this.level.push({t:this.selectedType,x:gx,y:gy});
  },
  bindInput(){
    let dragCamX=0, dragCamY=0;
    this.canvas.addEventListener('touchstart',e=>{
      e.preventDefault();
      if(this.isEditor && this.mode==='editor' && this.moveMode){
this.gesture.state='DRAG_CAM'; this.gesture.startX=e.touches[0].clientX;
this.gesture.startY=e.touches[0].clientY; dragCamX=this.camX; dragCamY=this.camY;
return; }
      if(e.touches.length===2 && this.isEditor){ this.gesture.state='DRAG_CAM';
this.gesture.startX=e.touches[0].clientX; this.gesture.startY=e.touches[0].clientY;
dragCamX=this.camX; dragCamY=this.camY; return; }
      if(this.isEditor && this.mode==='editor'){
```

```
      this.gesture.state='TOUCH_DOWN'; this.gesture.moved=false;
this.gesture.startX=e.touches[0].clientX; this.gesture.startY=e.touches[0].clientY;
this.gesture.startTime=Date.now();
      let r=this.canvas.getBoundingClientRect(); let wx=e.touches[0].clientX-r.left+this.camX,
wy=e.touches[0].clientY-r.top+this.camY;
      let ghost=document.getElementById('ghost'); ghost.style.display='block';
ghost.style.left=(Math.floor(wx/40)*40-this.camX)+'px';
ghost.style.top=(Math.floor(wy/40)*40-this.camY)+'px';
    } else { this.input.press=true; this.player.bufferTimer=100; }
  },{passive:false});
  this.canvas.addEventListener('touchmove',e=>{
    e.preventDefault();
    if(this.gesture.state==='DRAG_CAM'){ this.camX=Math.max(0,
dragCamX-(e.touches[0].clientX-this.gesture.startX)); this.camY=Math.max(-100,
Math.min(100, dragCamY-(e.touches[0].clientY-this.gesture.startY))); return; }
    if(this.gesture.state==='TOUCH_DOWN'){
      let dx=Math.abs(e.touches[0].clientX-this.gesture.startX),
dy=Math.abs(e.touches[0].clientY-this.gesture.startY);
      if(dx>12 || dy>12) this.gesture.moved=true;
      if(!this.gesture.moved){ let r=this.canvas.getBoundingClientRect(); let
wx=e.touches[0].clientX-r.left+this.camX, wy=e.touches[0].clientY-r.top+this.camY; let
ghost=document.getElementById('ghost');
ghost.style.left=(Math.floor(wx/40)*40-this.camX)+'px';
ghost.style.top=(Math.floor(wy/40)*40-this.camY)+'px'; }
      else document.getElementById('ghost').style.display='none';
    }
  },{passive:false});
  this.canvas.addEventListener('touchend',e=>{
    e.preventDefault(); document.getElementById('ghost').style.display='none';
    if(this.gesture.state==='DRAG_CAM'){ this.gesture.state='IDLE'; return; }
    if(this.gesture.state==='TOUCH_DOWN' && !this.gesture.moved &&
Date.now()-this.gesture.startTime<400){
      let r=this.canvas.getBoundingClientRect();
this.tryPlace(this.gesture.startX-r.left+this.camX, this.gesture.startY-r.top+this.camY);
    }
    this.gesture.state='IDLE'; this.input.press=false; this.input.pressConsumed=false;
this.triggers.clear();
  },{passive:false});
  this.canvas.addEventListener('mousedown',e=>{ if(e.pointerType==='touch') return;
if(this.isEditor && this.mode==='editor' && !this.moveMode){
this.tryPlace(e.clientX+this.camX, e.clientY+this.camY); } else if(!this.isEditor){
this.input.press=true; this.player.bufferTimer=100; } });
  this.canvas.addEventListener('mouseup',()=>{this.input.press=false;
this.input.pressConsumed=false;});
  this.canvas.addEventListener('contextmenu',e=>e.preventDefault());
  window.addEventListener('keydown',e=>{ if(e.code==='Space'){ this.input.press=true;
this.player.bufferTimer=100; } if(e.code==='KeyE' && this.mode==='editor')
```

```
this.toggleEditor(); if(e.code==='KeyM' && this.mode==='editor')
this.setMove(!this.moveMode); });
    window.addEventListener('keyup',e=>{ if(e.code==='Space'){this.input.press=false;
this.input.pressConsumed=false;}});
    document.getElementById('btnEdit').onclick=()=>this.toggleEditor();
document.getElementById('btnErase').onclick=()=>this.setErase(!this.eraseMode);
document.getElementById('btnSave').onclick=()=>this.save();
document.getElementById('btnMove').onclick=()=>this.setMove(!this.moveMode);
  },
  toggleEditor(){ if(this.mode!=='editor') return; this.isEditor=!this.isEditor;
document.getElementById('editTxt').innerText=this.isEditor?'EDITOR':'JUGANDO';
document.getElementById('btnEdit').classList.toggle('edit-active',this.isEditor);
document.getElementById('hint').innerText=this.isEditor?(this.moveMode?'✋ MOVER:
arrastrá con 1 dedo':'✏️ 1 toque = 1 bloque | 🚀 azul arriba = PUBLICAR'):'Toca para saltar';
if(!this.isEditor){ this.resetPlayer(); this.camX=0; this.camY=0; } },
  checkComplete(){
    if(this.hasCompleted) return; if(this.level.length<3) return;
    let maxX=Math.max(...this.level.map(o=>o.x)); let finishX=maxX+300;
    if(this.player.x > finishX){
      this.hasCompleted=true;
      if(this.mode==='editor'){ let s=DB.data.slots[this.currentSlot]; if(!s.verified){
s.verified=true; DB.save(); this.updatePublishButtons(); alert('✅ ¡VERIFICADO! Ahora el
botón 🚀 azul ya te deja publicar en la nube.'); } else alert('✅ ¡NIVEL COMPLETADO!');
this.restart(); return; }
      DETAIL.onComplete(this.currentPlayLevel);
      setTimeout(()=>{ if(confirm('🎉 ¡NIVEL COMPLETADO!
'+this.currentPlayLevel.name+'\n¿Volver a detalles?')){ UI.show('detailScreen');
DETAIL.open(this.currentPlayLevel); } else this.restart(); },100);
    }
  },
  checkPortals(p){
    if(this.portalCooldown>0) this.portalCooldown--;
    if(this.portalCooldown>0) return;
    for(let o of this.level){
      if(o.t!==6 && o.t!==7) continue;
      let crossed = p.prevX < o.x+40 && p.x >= o.x-10 && Math.abs((p.y+15) - (o.y+40)) < 120;
      let overlapped = p.x < o.x+50 && p.x+30 > o.x-10 && p.y < o.y+90 && p.y+30 > o.y-10;
      if(crossed || overlapped){
        if(o.t===6 && p.mode!=='CUBE'){ p.mode='CUBE'; p.vy=0;
document.getElementById('modeTxt').innerText='CUBO'; this.portalCooldown=15; break; }
        if(o.t===7 && p.mode!=='SHIP'){ p.mode='SHIP'; p.vy=0;
document.getElementById('modeTxt').innerText='NAVE'; this.portalCooldown=15; break; }
      }
    }
  },
  fixedUpdate(dt){
    if(this.isEditor) return; let p=this.player; p.prevX=p.x; p.prevY=p.y; p.x+=320*dt;
if(this.mode==='play') this.camX=p.x-150; let g=p.grav;
```

```
    if(p.bufferTimer>0) p.bufferTimer-=dt*1000; if(p.coyoteTimer>0) p.coyoteTimer-=dt*1000;
    let wasGround=p.onGround; p.onGround=false;
    if(p.mode==='CUBE'){ p.vy+=1400*g*dt; p.y+=p.vy*dt; if(g===1 && p.y>=350){ p.y=350;
p.vy=0; p.onGround=true; } if(g===-1 && p.y<=50){ p.y=50; p.vy=0; p.onGround=true; }
if(!p.onGround && wasGround) p.coyoteTimer=60; if(p.bufferTimer>0 && (p.onGround ||
p.coyoteTimer>0)){ p.vy=-620*g; p.bufferTimer=0; p.coyoteTimer=0; p.onGround=false; } }
    else { p.vy+=(this.input.press?-900:900)*g*dt; p.vy*=0.99; p.y+=p.vy*dt; }
    this.checkPortals(p);
    for(let o of this.level.filter(x=>Math.abs(x.x-p.x)<800)){
      let key=o.x+','+o.y+','+o.t; let overlap=p.x<o.x+40 && p.x+p.w>o.x && p.y<o.y+40 &&
p.y+p.h>o.y;
      if(!overlap){ this.triggers.delete(key); continue; }
      if(this.triggers.has(key) && [3,4,5,8,9,10,11,12].includes(o.t)) continue;
      if(o.t===1){ if(p.prevY+p.h<=o.y+5 && p.vy>=0 && g===1){ p.y=o.y-p.h; p.vy=0;
p.onGround=true; if(p.bufferTimer>0){ p.vy=-620*g; p.bufferTimer=0; } continue; }
if(p.prevY>=o.y+40-5 && p.vy<=0 && g===-1){ p.y=o.y+40; p.vy=0; p.onGround=true;
if(p.bufferTimer>0){ p.vy=620*g; p.bufferTimer=0; } continue; } if(p.y+p.h>o.y+8 &&
p.y<o.y+32){ this.die(); break; } }
      if(o.t===2){ if(p.x+22>o.x+10 && p.x+8<o.x+30 && p.y+p.h>o.y+15){ this.die(); break; } }
      if([3,4,5].includes(o.t) && this.input.press && !this.input.pressConsumed){ if(o.t===3)
p.vy=-420*g; if(o.t===4) p.vy=-620*g; if(o.t===5){ p.vy=-650*g; p.grav*=-1; }
this.input.pressConsumed=true; this.triggers.add(key); }
      if([10,11,12].includes(o.t)){ if(o.t===10) p.vy=-420*g; if(o.t===11) p.vy=-620*g;
if(o.t===12){ p.vy=-700*g; p.grav*=-1; } this.triggers.add(key); }
      if(o.t===8 && p.grav!==1){ p.grav=1; this.triggers.add(key); }
      if(o.t===9 && p.grav!==-1){ p.grav=-1; this.triggers.add(key); }
    }
    if(p.y<-100||p.y>500) this.die();
    this.checkComplete();
  },
  draw(){
    if(!document.getElementById('gameScreen').classList.contains('show')) return;
    let ctx=this.ctx; ctx.clearRect(0,0,this.canvas.width,this.canvas.height); ctx.save();
ctx.translate(-this.camX, -this.camY);
    let viewLeft=this.camX-50, viewRight=this.camX+this.canvas.width+50,
viewTop=this.camY-50, viewBottom=this.camY+this.canvas.height+50;
    ctx.strokeStyle='#ffffff0a'; for(let i=Math.floor(viewLeft/40)*40;i<viewRight;i+=40){
ctx.beginPath(); ctx.moveTo(i,viewTop); ctx.lineTo(i,viewBottom); ctx.stroke(); }
    for(let o of this.level.filter(x=>x.x>viewLeft && x.x<viewRight && x.y>viewTop &&
x.y<viewBottom)){
      ctx.fillStyle=TYPES[o.t].c; if(o.t===1) ctx.fillRect(o.x,o.y,40,40); else if(o.t===2){
ctx.beginPath(); ctx.moveTo(o.x,o.y+40); ctx.lineTo(o.x+20,o.y); ctx.lineTo(o.x+40,o.y+40);
ctx.fill(); } else if(o.t<=5){ ctx.beginPath(); ctx.arc(o.x+20,o.y+20,16,0,Math.PI*2); ctx.fill(); }
else if(o.t>=10){ ctx.fillRect(o.x,o.y+30,40,10); } else{ ctx.fillRect(o.x,o.y,40,80); }
    }
    if(!this.isEditor){ ctx.fillStyle=this.player.mode==='CUBE'?'#00ff88':'#ffcc00';
ctx.fillRect(this.player.x,this.player.y,this.player.w,this.player.h); }
    ctx.restore();
```

```
  },
  loop(t){ let ft=Math.min((t-this.lastTime)/1000,0.1); this.lastTime=t; this.acc+=ft;
while(this.acc>=1/240){ this.fixedUpdate(1/240); this.acc-=1/240; } this.draw();
requestAnimationFrame(tt=>this.loop(tt)); }
};
DB.load(); UI.renderMenu(); GAME.init();
CLOUD.fetchAll();
</script>
</body>
</html>
```